BetaSix: Multi-task NNUE Evaluation and Efficient Alpha-Beta Search for 19×19 Connect6

Author:Boxian Baiyang

BetaSix Developer:Boxian Baiyang

BetaSix Training:Boxian Baiyang

Review and proofreading:Boxian Baiyang

Acknowledgements: The model training task of this work was executed with the AI tool MiMo Claw, and the manuscript review was assisted by the AI tools MiMo-V2.6-Pro (both developed by Xiaomi LLM-Core). The generation of training data, experimental design, and final responsibility for the content of the paper lie with the author.

Abstract: Connect6 is played on a 19×19 board where each move places two stones, yielding a branching factor and tactical combinatorics far beyond Gomoku and imposing stricter demands on position evaluation and search efficiency. This paper presents BetaSix, an efficient neural network game engine for 19×19 Connect6. BetaSix employs a compact 736→256→64→1 multi-layer perceptron as its NNUE evaluation trunk, taking 722 board-plane dimensions together with 14 handcrafted tactical features as input. Following the multi-task learning idea of KataGo, five heads—value, win/draw/loss (WDL), moves-to-end (TTE), ownership, and policy—are trained jointly to suppress shortcut overfitting to a single scalar objective. To the best of our knowledge, BetaSix is the first **complete open-source NNUE-based Connect6 engine for general play, with trained weights released**(Note: hzyhhzy's KataGomo project released NNUE evaluation code for VCF solving (with both α-β and MCTS variants) in its ConnectSix_NNUE_VCF branch; however, no trained weights were released and it does not constitute a complete engine for general play.). Adopting the incremental-update idea of Rapfi, placing and removing stones touch only the corresponding column of the first-layer accumulator, reducing leaf-evaluation cost in depth-first alpha-beta search by roughly two orders of magnitude. The search side combines iterative deepening, a Zobrist transposition table, null-move pruning, LMR, and cooperative two-stone move generation. On a self-constructed benchmark of 100 tactical positions in five categories (deep tactics, complex midgame, traps, endgame mates, and counter-attacks), the final model trained on 1.1M self-play samples reaches a pass rate of 90/100, a clear improvement over earlier versions (84, 85), with 19/20 on counter-attack positions. The paper also systematically summarizes seven critical defects found and fixed during training and debugging, offering reusable engineering experience for connection-game AIs.

Keywords: Connect6; neural network evaluation; NNUE; multi-task learning; alpha-beta search; incremental update

1 Introduction

Game playing has long been a touchstone of artificial intelligence. From chess and shogi to Go, the combination of deep reinforcement learning and Monte-Carlo tree search (Silver et al., 2016; 2017a; 2017b; Schrittwieser et al., 2020) keeps raising the ceiling of game AI, but such methods typically require high-end GPUs and massive computation. Meanwhile, deploying strong game AIs in accelerator-free, time-constrained environments remains a practical need: engines for mobile, web, and teaching scenarios must respond instantly on ordinary CPUs. Rapfi (Jin et al., 2025) showed on Gomoku that a compact network built with "distillation + incremental update" can outperform large AlphaZero-based engines without a GPU. This paper carries that idea into Connect6, with its more complex rules and larger search space.

Connect6 was proposed by I-C. Wu in 2005. Played on a 19×19 board with Black moving first, Black places one stone on the first move and both sides place two stones per move thereafter; the first player to connect six in a row horizontally, vertically, or diagonally wins. Compared with one-stone-per-move Gomoku, the two-stone rule introduces two fundamental complications. First, a move is a pair of points rather than a single point, so the number of legal moves grows quadratically and candidate pruning with pairwise scoring becomes necessary. Second, "two stones per move" changes the move-count semantics of threats such as open fours and open fives (an open four is a win in one move), and two-stone cooperation producing layered attacks (double threats, forced wins) becomes decisive, requiring the evaluation and search to be sensitive to cooperation rather than single-point value. On the other hand, Connect6 is symmetric and has no forbidden moves; the first-mover advantage is milder than in Gomoku, and frequent attack-defense transitions demand stronger modeling of defensive signals.

To address these challenges, we propose the BetaSix engine, whose core has three parts. (1) Compact NNUE evaluation: a 736-dimensional input (722 board-plane dimensions plus 14 handcrafted tactical features) passes through a 736→256→64→1 ReLU multi-layer perceptron to output a position value, with about 205K parameters and roughly 0.2M multiply-accumulates per evaluation. (2) Multi-task distillation training: five heads—value, WDL, TTE, ownership, and policy (KataGo-style auxiliary tasks, Wu, 2019)—are attached to the shared trunk and trained jointly on 1.1M samples generated by depth-limited self-play, markedly improving the learning of attack-defense semantics. (3) Incremental update and efficient search: placing a stone updates only the corresponding column of the first-layer accumulator, and together with alpha-beta iterative deepening, a transposition table, null-move pruning, and LMR, this reaches practically usable depths on ordinary CPUs.

Our main contributions are as follows:

- A compact NNUE evaluation architecture for 19×19 Connect6 (736→256→64→1) that augments pure board planes with 14 handcrafted tactical features for threat semantics, with about 205K parameters and about 0.2M FLOPs per forward pass;
- A five-head (value/WDL/TTE/ownership/policy) multi-task loss distilling a shared trunk, where auxiliary tasks prevent the network from degenerating into a single-scalar shortcut solution and yield better generalization on attack, defense, and endgame semantics;

- A Rapfi-style incremental update: each place/unplace during depth-first search touches one column of the first-layer accumulator (O(H1)), about two orders of magnitude cheaper than a full forward pass (O(736·H1)), together with its integration with alpha-beta search;
- A 100-position five-category tactical benchmark (benchmark_v2) and a reproducible self-play training pipeline; the final model passes 90/100 with 19/20 on counter-attacks, and the diagnosis and repair of seven critical defects are documented. To the best of our knowledge, BetaSix is the first complete open-source NNUE-based Connect6 engine for general play, with trained weights released

2 Related Work

2.1 Efficient Neural Networks

Improving network efficiency under limited computation is a long-standing theme. MobileNets (Howard et al., 2017; Sandler et al., 2018; Howard et al., 2019) cut computation with depth-wise separable convolutions; SENet (Hu et al., 2018), SKNet (Li et al., 2019), CondConv (Yang et al., 2019), and dynamic convolution (Chen et al., 2020) raise the expressive power per FLOP via attention or conditional parameterization. In game playing, NNUE (Nasu, 2018) pioneered the "feature difference + shallow fully-connected network" evaluation paradigm, widely adopted by engines such as Stockfish; Rapfi (Jin et al., 2025) further proposed pattern-codebook distillation and incremental updates, amortizing local pattern feature computation to nearly zero. BetaSix follows the NNUE/Rapfi line, but restructures features and evaluation semantics for the two-stone rules of Connect6 and replaces single value regression with multi-task distillation.

2.2 Game-Tree Search Algorithms

The classical game-tree method is alpha-beta pruning (Knuth & Moore, 1975), augmented by null-move pruning (Donninger, 1993), futility pruning (Heinz, 1998), LMR and razoring (Hoki & Muramatsu, 2012), and the history heuristic (Schaeffer, 1983). Monte-Carlo tree search (Coulom, 2006) and its UCT (Gelly & Wang, 2006) and RAVE (Rimmel et al., 2010) variants dominate in Go. For connection games, threat-space search (Allis et al., 1993) and proof-number search (Allis, 1994) are extremely effective on tactically won positions. BetaSix takes the alpha-beta route: its evaluation is cheap and incrementally updatable, matching depth-first traversal naturally, while a gap-aware six-window threat scan, inspired by threat-space search, strengthens tactical signals.

2.3 Agents for Connection Games

Recently, hzyhhzy released the KataGomo Six-in-a-Row model (b18) based on the KataGo reinforcement learning pipeline. Its ConnectSix_NNUE_VCF branch also explored NNUE evaluation components for VCF win-search (unreleased weights). The difference between BetaSix and KataGomo lies in their approaches: KataGomo uses MCTS plus a large-scale convolutional network (93.6MB, GPU-dependent), while BetaSix employs NNUE plus an α-β search route (5.03MB, real-time pure CPU), targeting deployment scenarios without accelerators.

Gomoku has been proven a first-player win (Allis et al., 1996); optimal play from balanced openings remains hard. Methods include threat-space search (Allis et al., 1993), adaptive dynamic programming (Zhao et al., 2012), genetic algorithms (Wang & Huang, 2014), and AlphaZero-style methods (Wu, 2019; Xie et al., 2018; Wang, 2018). Rapfi ranked first among 520 Gomoku agents on Botzone and won GomoCup 2024, validating the ceiling of the "lightweight network + finely tuned search" route. For Connect6, engines such as NCTU6 excel at shape-based scoring and two-stone move generation;to the best of our knowledge, BetaSix is the first complete open-source NNUE-based Connect6 engine for general play, with trained weights released , providing a complete training–evaluation–deployment loop.

3 Method

This section presents the design of BetaSix: we first give a system overview, then describe the input representation and handcrafted tactical features, the multi-task network and distillation training, the incremental update mechanism, and finally evaluation fusion and search.

3.1 System Overview

As shown in Figure 1, BetaSix consists of an evaluation module and a search module. The evaluation module feeds the 736-dimensional input (722 board-plane dimensions plus 14 handcrafted tactical features) through a three-layer fully-connected trunk to output a scalar value; during training, five task heads are mounted on the trunk and trained jointly, while only the value head is kept at inference. The search module traverses the game tree with alpha-beta iterative deepening; leaf nodes take a weighted fusion of the handcrafted shape score and the NNUE output. Move generation uses "candidate pruning + two-stone pairwise scoring" to fit the two-stones-per-move rule.

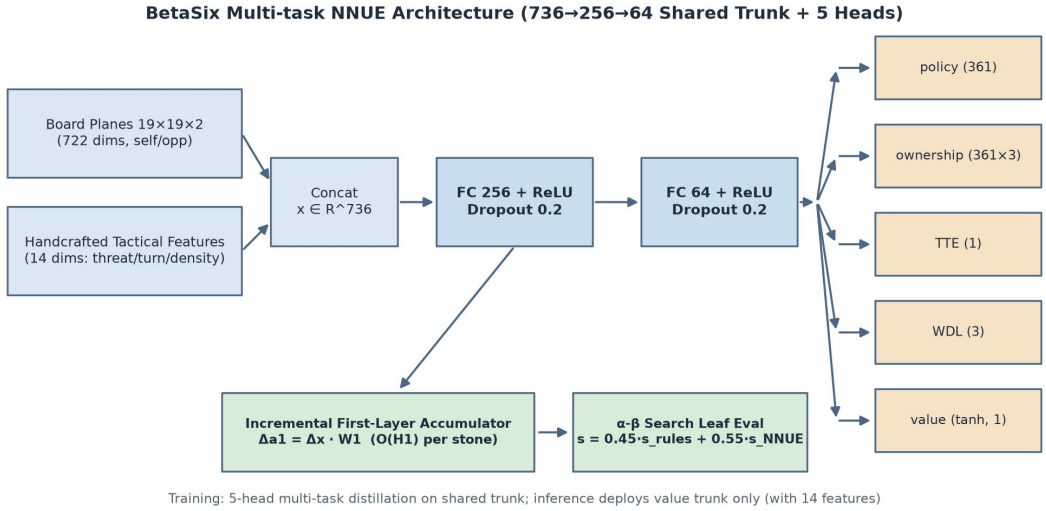


Figure 1: BetaSix system and network architecture. The input combines 722 board-plane dimensions and 14 handcrafted tactical features; after the 736→256→64 trunk, five task heads produce outputs. Inference deploys only the value head, together with incremental update and alpha-beta search.

3.2 Input Representation and Handcrafted Tactical Features

The board is represented as two 19×19=361 binary planes—own stones and opponent stones, normalized to the side to move—for 722 dimensions in total. Pure board planes leave the tactical semantics of open threes, open fours, and double threes implicit, and the network would need massive samples to learn them. We therefore add 14 explicit handcrafted tactical features (Table 1). For each direction (horizontal, vertical, and two diagonals), we count consecutive runs of each side by length and number of open ends, and aggregate counts of open2, open3, sleep3, open4, and double3 under the sub-run merging rule; we then append the side to move, stone density, and two key defensive signals (whether the opponent threatens a win, and whether the opponent has an open four).

Table 1: Definition of the 14 handcrafted tactical features (f0–f4 are opponent threats, f5–f9 own threats, consistent with the training-data convention).

Index	Feature	Description
f0–f4	Opponent open2 / open3 / sleep3 / open4 / double3	Counts of opponent threat runs
f5–f9	Own open2 / open3 / sleep3 / open4 / double3	Counts of own threat runs
f10	Side to move	1 = Black to move, 0 = White to move
f11	Stone density	Stones placed / 361
f12	opp_threatens_win	Opponent has an open four or two-plus open threes
f13	opp_has_open4	Opponent has an open four (win-in-one threat)

Feature statistics use the sub-run merging rule: a run of length n contributes 1 open n, n−1 open n−1, …, and n−1 open2, so a long run naturally contributes all of its sub-threats without omissions. The defensive signals f12/f13 are taken from the opponent perspective; this convention must be strictly consistent between training-data generation and engine inference (see the encoding defect analysis in Section 5.4).

3.3 Multi-task Network and Distillation Training

The trunk is a three-layer fully-connected network 736→256→64→1 with ReLU between layers and dropout 0.2 during training. The output value is compressed to (−1,1) by tanh and mapped to the engine score space by a scale S=200000:

$$\hat{v} = tanh(W_3 \cdot ReLU(W_2 \cdot ReLU(W_1 x + b_1) + b_2) + b_3) \cdot S, \quad S = 200000 \quad (1)$$

Pure value regression easily lets the network take shortcuts—learning a crude scalar correlated with the outcome while ignoring attack-defense structure. Following the multi-task design of KataGo (Wu, 2019), we attach five heads to the shared trunk: value (a tanh scalar), WDL (three-way win/draw/loss), TTE (moves-to-end regression), ownership (361 points × 3 classes), and policy (361-way move classification). The total loss is:

$$L = L_value + 0.5 \cdot L_WDL + 0.3 \cdot L_TTE + 0.3 \cdot L_own + 0.7 \cdot L_policy \quad (2)$$

where L_value is MSE (labels are first normalized to the side to move and mean-centered to remove the first-player bias), L_WDL is KL divergence, L_TTE is MSE on normalized moves-to-end, L_own is weighted cross-entropy (weight 1.0 for occupied points and 0.1 for empty points, mitigating class imbalance), and L_policy is move cross-entropy. Auxiliary tasks—especially policy and ownership—

force the trunk to encode rich board understanding and alleviate single-scalar degeneration. Table 2 lists the parameters per layer: the trunk and value head total about 205K parameters and about 0.2M multiply-accumulates per forward pass, suitable for real-time CPU evaluation.

Table 2: Network parameters and computation (the inference path contains only the trunk and the value head).

Component	Shape	Parameters	MACs
W1 / b1	736 × 256 / 256	188,672	188,416
W2 / b2	256 × 64 / 64	16,448	16,384
W3 / b3	64 × 1 / 1	65	64
Total (inference)	—	205,185	≈ 0.2M
Five heads (training only)	value/WDL/TTE/own/pol	≈ 77K	—

3.4 Incremental Update

Alpha-beta search is depth-first: each move changes only two stones on the board and restores them on backtracking. Recomputing the first layer $z = W_1 x + b_1$ in full at every leaf would redo 732 unchanged input dimensions. Since the first layer is linear in the board planes, placing a stone equals adding (or subtracting) a one-hot column to the input, so the accumulator can be maintained incrementally:

$$z \leftarrow z \pm W_1[\cdot, p], \quad p \text{ indexes the input dimension of the placed point} \qquad (3)$$

That is, each place/unplace only adds or subtracts one 256-dimensional column (O(H1)), whereas a full forward pass costs O(736·H1). Since the 14 handcrafted features vary nonlinearly with the position, they are recomputed at each forward pass and folded in separately (14×256 multiply-accumulates, a small share). Figure 2 illustrates the mechanism: in a typical search, placements far outnumber leaves, so incremental update raises evaluation throughput by about two orders of magnitude, consistent with the observation of Rapfi on Gomoku. In implementation, the engine maintains the accumulator g_nnA1[256]; put()/undo call the incremental update, and a full-rebuild interface guarantees that the accumulator stays strictly in sync with the board when switching networks or side-to-move perspectives.

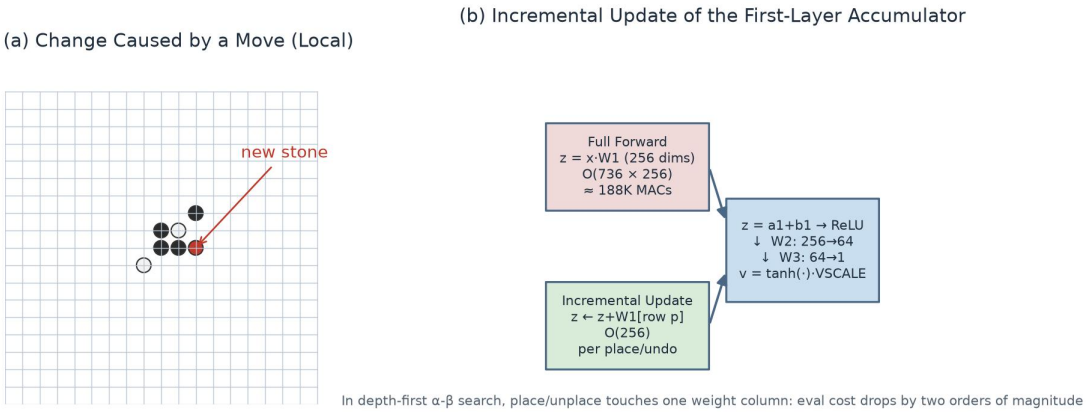


Figure 2: Incremental update. Left: one placement changes only one input dimension. Right: cost comparison between a full forward pass and incremental update, plus the forward path of the following layers.

3.5 Evaluation Fusion and Search

Evaluation fuses the handcrafted shape score with the NNUE output (a Rapfi-style "classical + network" hybrid):

$$s(\sigma) = (1 - \alpha)\cdot s_rules(\sigma) + \alpha\cdot s_NNUE(\sigma), \quad \alpha = 0.55 \quad (4)$$

s_rules is a symmetric zero-sum shape score: maximal runs (length L, open ends o) are scored by table (e.g., open four 26000, open five 420000, sleep five 80000), and multiple threats such as double open fours receive fork bonuses on the order of 7,000,000. A gap-aware six-window scan additionally scores gapped threats with ≥4 same-colored stones and no opponent stone in the window (e.g., XX_XX) at 26000 each, and windows one stone short of six at 3,000,000 each, so the search actively blocks rather than relying on static rules. The two sides use strictly symmetric weights to preserve the zero-sum property. The NNUE weight α=0.55 makes the network dominant, but candidate move ranking still uses only the handcrafted score, preventing network noise from pushing critical defensive points out of the candidate window (this design proves essential to defensive correctness in practice).

Move generation is designed for "two stones per move": empty points within three cells of existing stones form the candidates (the central neighborhood on an empty board); the top-K (K=12) candidates by handcrafted score are paired into moves, and each pair is scored as the sum of the two point scores plus a synergy bonus (1200 for Manhattan distance ≤2, 900 for sharing a row/column/diagonal), encouraging the two stones to build together rather than being isolated good points. The search is negamax alpha-beta: root iterative deepening (depth- and time-capped), a Zobrist transposition table (2^18 entries with exact/lower/upper flags), null-move pruning ($R = 2 + \lfloor depth/3 \rfloor$, enabled at depth ≥ 3), LMR (depth ≥ 3 and from the 4th move on, reducing 1–2 plies with a full-window re-search on failure), and the root best move is updated only when alpha improves (fixing an early iterative-deepening regression defect).

4 Experimental Setup

4.1 Dataset

Training data is generated by self-play (Figure 3): the generator drives both sides with depth-limited search, writing one 790-byte record per position, containing board(361) + value(4) + wdl(1) + tte(4) + polr/polc(2) + mover(1) + 14 features(56) + ownership(361). The value label is the deeper search evaluation (Black perspective, flipped by the side to move at replay), and wdl/tte/ownership are back-filled from game outcomes. Table 3 lists the sample sizes: after fixing the rule and feature conventions, two old batches (about 960K records in total) were discarded, and regeneration and merging yielded 1,101,585 valid samples (r5000 292,990 + d1m 425,847, etc.).

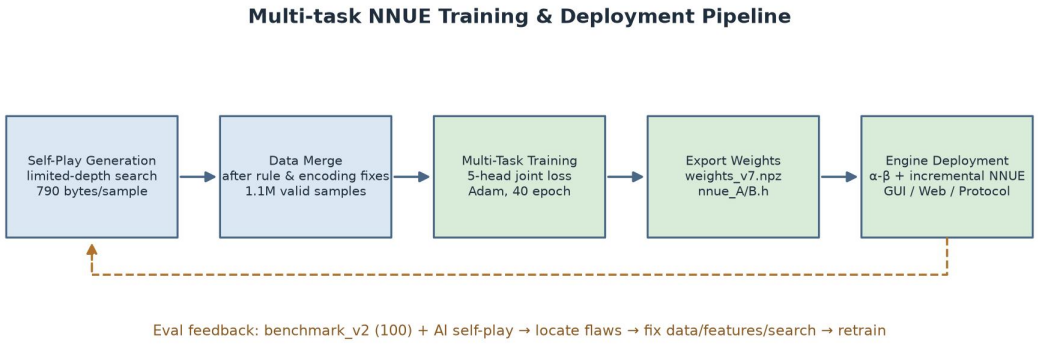


Figure 3: Training and deployment pipeline. Self-play generates 790-byte records; after merging, multi-task training produces weights that are loaded into the engine, and evaluation feedback drives data and feature iterations.

Table 3: Evolution of training-data size

Batch	Samples	Status
d5000	510,788	Discarded (first-move rule error)
d750k	769,123	Discarded (first-move rule error)
r5000	292,990	Valid
d1m	425,847	Valid
Merged total	1,101,585	Final training set

4.2 Training Details

Training uses PyTorch (CPU, 2 threads): Adam (lr=1e-3, weight_decay=1e-4), batch size 2048, up to 40 epochs, an 80/20 train/validation split, and early stopping when the validation loss rises for 3 consecutive rounds. Value labels are mean-centered, TTE is normalized by 70, and weighted cross-entropy for ownership mitigates empty-point dominance. The data loader uses memmap with random sampling (up to 60K records per run) to avoid OOM on large files. After training, float weights (weights_v*.npz) are exported and converted to C headers (nnue_A.h/nnue_B.h) for the engine, while an int8 quantized export path is kept for deployment optimization. Table 4 summarizes the main hyperparameters.

Table 4: Training hyperparameters

Hyperparameter	Value	Hyperparameter	Value
Trunk	736→256→64→1	Dropout	0.2
Optimizer	Adam	LR / weight decay	1e-3 / 1e-4
Batch size	2048	Max epochs	40
Loss weights (v/w/t/o/p)	1 / 0.5 / 0.3 / 0.3 / 0.7	Value scale S	200,000
Train/val split	80/20	Early stop	val loss rises 3 rounds

4.3 Evaluation Protocol

The main evaluation uses our benchmark benchmark_v2: 100 positions in five categories of 20 each—deep tactics (wins in 3–5 moves), complex midgame (both sides have threats), traps (the surface move differs from the correct one), endgame mates (one-move mate recognition), and counter-attack

(defend then counter). Ground-truth answers come from depth-10 search (1500ms), and the model under test answers with depth-5 search (400ms); a pass requires the move pair to match the ground truth (order of the two stones exchanged allowed). This protocol measures the chess sense of the evaluation network as "strong search as teacher, shallow search as student", better reflecting the strength of the evaluation function itself than win-rate alone. Auxiliary evaluations are AI self-play between different weights of the same engine (10 games) and per-category attack-defense statistics.

5 Results

5.1 Benchmark Results

Table 5 shows the pass rates of each version on benchmark_v2. The final version weights_v7 (1.1M samples + feature-encoding fix) reaches 90/100, with category scores in Figure 4(b): deep tactics 14/20, complex midgame 19/20, traps 18/20, endgame mates 20/20, and counter-attacks 19/20. The perfect endgame-mate score and the high counter-attack score indicate that multi-task training and defensive features model attack-defense semantics effectively.

Table 5: Pass rates on benchmark_v2 (100 positions; depth-5 answers vs depth-10 ground truth)

Model	Training Samples	Total	Deep Tactics	Complex Midgame	Trap	Endgame Mate	Counter-Attack
weights_v5	670K	84	—	—	—	—	—
weights_v6	1.1M	85	—	—	—	—	—
weights_v7 (final)	1.1M + encoding fix	90	14/20	19/20	18/20	20/20	19/20
Wide 512/256/128	1.1M	66	4/20	—	—	—	—

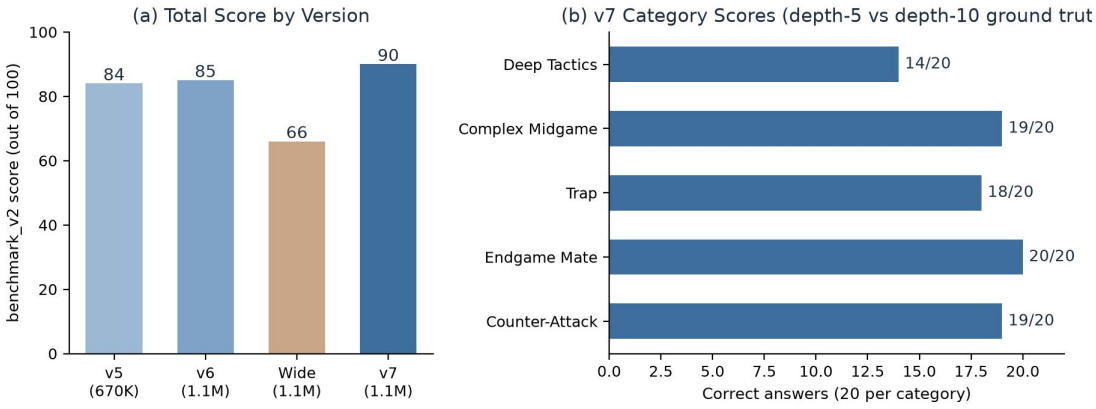


Figure 4: benchmark_v2 results. (a) Total scores by version: v7 reaches 90/100 while the wide-net variant degrades clearly. (b) Category scores of v7: perfect on endgame mates, 19/20 on counter-attacks.

5.2 Version Evolution and Data Scale

Figure 4(a) and Table 5 jointly depict the version evolution: v5 (670K samples) 84 → v6 (1.1M samples) 85 → v7 (encoding fix) 90. Doubling the data brought only 1 point, whereas the feature-encoding fix brought 5 points, indicating that beyond data scale, the consistency of feature conventions between training and inference is the real bottleneck. The wide net (736→512→256→128→1) scored

only 66 on the same data (deep tactics 4/20), showing that widening the network at 1.1M samples aggravates overfitting and tactical underfitting; we therefore rolled back to the narrow architecture.

5.3 Self-Play and Attack-Defense Analysis

In 10 self-play games between two weights of the same engine, v6 beat v5 6–4. Game records show that endgames are dominated by long-range attack-defense transitions (e.g., the game fragment in Appendix A), confirming the importance of "cooperative kills" under the two-stone rule. In category statistics, 19/20 on counter-attacks contrasts sharply with the clear degradation before the encoding fix: before the fix, the perspective conventions of f12/f13 were opposite between training data and engine inference, systematically swapping the defensive signals; in practice this appeared as "attack without defense" and massive losses on benchmark defensive positions.

5.4 Ablations and Engineering Lessons

Beyond structural ablations, we document seven critical defects found and fixed during the training–debugging cycle (Table 6). The three most impactful are: (1) the unconditional assignment of the best move in iterative deepening completely blocked the policy signal of the multi-task network (0% kill capture), fixed to reach 60%; (2) inconsistent feature encoding between data generation and engine inference (OPP/SELF perspectives swapped) directly caused the defensive weakness, and the fix raised the benchmark from 85 to 90; (3) a rule error (generating 2 stones on the first move instead of 1) wasted about 960K samples. These defects jointly suggest that in neural game systems, the "interface consistency" of data pipelines, feature conventions, and search logic is more likely to become a bottleneck than the network architecture itself.

Table 6: Diagnosis and repair of the seven critical defects

No.	Defect	Root Cause	Fix & Effect
1	Best-move regression	Unconditional iterBest assignment	Update only on alpha rise; kills 0%→60%
2	Asymmetric evaluation	Unequal side weights (+1188/−1188, etc.)	Symmetrized; empty board ±622
3	WDL direction reversed	Labels were absolute winners	Flip by side to move; KL loss
4	First-move rule error	Always generated 2 stones	1 stone on move 1; 960K samples discarded
5	Feature encoding mismatch	f12/f13: data used OPP, engine SELF	Engine switched to OPP; benchmark 85→90
6	Six-check color-code error	0/1 side flag vs 1/2 board code	has6(colour+1); self-play misjudgment gone
7	Macro & code-block misplacement	Wrong NN_H2 macro; leftover block	Explicit NN_H1/H2; 256/64 restored

5.5 Limitations

Our evaluation has the following limitations, which should be considered when interpreting the results. First, the benchmark is self-constructed: the 100 positions of benchmark_v2 were built by this project (ground truth from depth-10 search) rather than a publicly recognized standard test set, limiting cross-paper and cross-engine comparability. Second, there is no external Elo rating: we have not run

large-scale ladder matches against external engines such as Katagomo or NCTU6, so our strength claims are limited to the agreement between depth-5 answers and the depth-10 teacher (90/100) and self-play comparisons between different weights of the same engine, without absolute strength calibration. Third, the self-play sample is small: the version comparison (v6 beat v5 6–4) covers only 10 games, which is insufficient for statistically significant conclusions and serves only as a directional signal. To address these gaps, future work will add ladder matches against external engines with Elo ratings, expand and release the benchmark, and continue scaling the training data.

6 System Implementation

BetaSix exposes its core through a C ABI (b6_new_game / b6_play / b6_clear / b6_cell / b6_search / b6_set_net / b6_set_perspective / b6_eval_value / b6_opening), easing multiple packaging forms. The release provides two forms. (1) Command-line engine: communicating over a line-based stdin/stdout protocol (START / PING / RESET / BOARD <361 chars> <side> / GO <timelimit> <depth> / QUIT); every thinking call rebuilds the position from the BOARD payload, guaranteeing by construction that undo and restart keep engine and UI states consistent; GO returns one move of two stones, and the first move returns a single stone per the rules. Binaries are built with g++ -O2 -msse4.1 for Linux and Windows (static mingw). (2) GUI and web: a Python/Tkinter interface drives the engine as a subprocess with timeout protection; the web version is compiled with Emscripten to asm.js (no wasm dependency) and runs in any modern browser with bundled weights. The engine supports hot-switching between two networks (nnue_A new / nnue_B old) for A/B comparison and rollback.

7 Conclusion

This paper presents BetaSix, an efficient engine for 19×19 Connect6: a compact 736→256→64→1 NNUE at the core of evaluation, five-head multi-task distillation suppressing shortcut overfitting, incremental update cutting alpha-beta evaluation cost by about two orders of magnitude, and cooperative two-stone move generation with classical search enhancements, reaching 90/100 on a 100-position tactical benchmark (19/20 on counter-attacks) and delivered in full engineering forms (command line, GUI, web). To the best of our knowledge, BetaSix is the first complete open-source NNUE-based Connect6 engine for general play, with trained weights released, providing a reusable reference implementation of evaluation–search co-design for connection games under constrained compute.

Our conclusions are subject to the limitations in Section 5.5: the evaluation rests on a self-built benchmark, lacks external Elo calibration, and uses a small self-play sample. Future work includes: (1) ladder matches against external engines with Elo ratings, and expanding and releasing the benchmark; (2) scaling data beyond the current 1.1M and raising label search depth; (3) adopting residual or convolutional structures rather than plain widening, as the wide-net experiment suggests; (4) completing full int8 quantized deployment and evaluating the speed–accuracy trade-off; and (5) exploring threat-space search and proof-number search for endgame mate detection to further raise deep-tactics scores.

References

[1] Allis, L. V., Herik, H. J. v. d., and Huntjens, M. P. H. Go-moku and threat-space search. University of Limburg, 1993.

[2] Allis, L. V. Searching for solutions in games and artificial intelligence. Ponsen & Looijen Wageningen, 1994.

[3] Alus, L. V., Herik, H. J. v. d., and Huntjens, M. P. H. Go-moku solved by new search techniques. Computational Intelligence, 12(1):7–23, 1996.

[4] Chen, Y., Dai, X., Liu, M., Chen, D., Yuan, L., and Liu, Z. Dynamic convolution: Attention over convolution kernels. In CVPR, pp. 11030–11039, 2020.

[5] Coulom, R. Efficient selectivity and backup operators in Monte-Carlo tree search. In Computers and Games, pp. 72–83, Springer, 2006.

[6] Donninger, C. Null move and deep search. ICGA Journal, 16(3):137–143, 1993.

[7] Gelly, S. and Wang, Y. Exploration exploitation in Go: UCT for Monte-Carlo Go. In NIPS Workshop, 2006.

[8] Heinz, E. A. Extended futility pruning. ICGA Journal, 21(2):75–83, 1998.

[9] Hoki, K. and Muramatsu, M. Efficiency of three forward-pruning techniques in Shogi. Entertainment Computing, 3(3):51–57, 2012.

[10] Howard, A. G. et al. MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861, 2017.

[11] Hu, J., Shen, L., and Sun, G. Squeeze-and-excitation networks. In CVPR, pp. 7132–7141, 2018.

[12] Jin, Z., Duan, H., and Hang, Z. Rapfi: Distilling efficient neural network for the game of Gomoku. arXiv:2503.13178, 2025.

[13] Knuth, D. E. and Moore, R. W. An analysis of alpha-beta pruning. Artificial Intelligence, 6(4):293–326, 1975.

[14] Ma, X., Dai, X., Bai, Y., Wang, Y., and Fu, Y. Rewrite the stars. In CVPR, pp. 5694–5703, 2024.

[15] Nasu, Y. Efficiently updatable neural-network-based evaluation functions for computer Shogi. The 28th World Computer Shogi Championship Appeal Document, 185, 2018.

[16] Rimmel, A., Teytaud, F., and Teytaud, O. Biasing Monte-Carlo simulations through RAVE values. In Computers and Games, pp. 59–68, Springer, 2010.

[17] Schaeffer, J. The history heuristic. ICGA Journal, 6(3):16–19, 1983.

[18] Schrittwieser, J. et al. Mastering Atari, Go, chess and Shogi by planning with a learned model. Nature, 588(7839):604–609, 2020.

[19] Silver, D. et al. Mastering the game of Go with deep neural networks and tree search. Nature, 529(7587):484–489, 2016.

[20] Silver, D. et al. Mastering chess and Shogi by self-play with a general reinforcement learning algorithm. arXiv:1712.01815, 2017a.

[21] Silver, D. et al. Mastering the game of Go without human knowledge. Nature, 550(7676):354–359, 2017b.

[22] Wu, D. J. Accelerating self-play learning in Go. arXiv:1902.10565, 2019.

[23] Wu, I-C. Introducing Connect6. ICGA Journal, 28(4), 2005.

[24] Xie, Z., Fu, X., and Yu, J. AlphaGomoku: An AlphaGo-based Gomoku artificial intelligence using curriculum learning. arXiv:1809.10595, 2018.

[25] Zhao, D., Zhang, Z., and Dai, Y. Self-teaching adaptive dynamic programming for Gomoku. Neurocomputing, 78(1):23–29, 2012.

[26] hzyhhzy. KataGomo: Multi-game engine based on KataGo (ConnectSix2024 / ConnectSix_NNUE_VCF branches). GitHub repository, 2022–2026. https://github.com/hzyhhzy/KataGomo

Appendix A Self-Play Game Record Sample

The following is a fragment of a self-play game (game_01). Coordinates use "letter column + digit row" notation, with "+" joining the two stones of each move; the first move places one stone per the Connect6 rules.

G1 / H1+E2 / F1+J1 / E1+D1 / G2+K1 / D2+L1 / K2+C3 / C2+C1 / B2+B1 / L2+K3 / A1+D4 / E5+E3 / E4+F4 / F3+G4 / C4+H5 / B4+J4 / I5+G3 / M1+J5 / H2+J2 / I2+K4 …

The game lasted over 80 moves, with repeated double-open-three threats and counter-threats in the midgame; Black eventually won with a layered cooperative attack, illustrating the fast attack-defense transitions and rich combinations of the two-stones-per-move rule.

October 23, 2026